

PowerTrip: Exploiting Federated Heterogeneous Datacenter Power for Distributed ML Training

Talha Mehboob
University of Massachusetts Amherst
tmehboob@umass.edu

Luanzheng Guo
Pacific Northwest National Lab
lenny.guo@pnnl.gov

Nathan Tallent
Pacific Northwest National Lab
nathan.tallent@pnnl.gov

Michael Zink
University of Massachusetts Amherst
mzink@cas.umass.edu

David Irwin
University of Massachusetts Amherst
irwin@ecs.umass.edu

Abstract

The exponential growth of large-scale AI models has led to computational and power demands that can exceed the capacity of a single data center. This is due to the limited power supplied by regional grids that leads to limited regional computational power. Consequently, distributing training workloads across geographically distributed sites has become essential. However, this approach introduces a significant challenge in the form of communication overhead, creating a fundamental trade-off between the performance gains from accessing greater aggregate power and the performance losses from increased network latency. Although prior work has focused on reducing communication volume or using heuristics for distribution, these methods assume constant homogeneous power supplies and ignore the challenge of heterogeneous power availability between sites.

To address the challenge of training large models in power-constrained, geo-distributed environments, we introduce PowerTrip, a system that dynamically selects a subset of sites during runtime to optimize the power-communication trade-off. Specifically, PowerTrip selects sites based on a power-to-cost heuristic, prioritizing those with high power availability and low network latency. PowerTrip employs a dynamic greedy approach and uses the marginal gain in training efficiency, i.e., accuracy improvement per unit of time, to optimize for the number of sites where the performance penalty from network overhead negates the benefit of adding more computational power. Our evaluation, which uses real-world Google power traces to model realistic power capacity constraints, demonstrates that PowerTrip can reduce time-to-accuracy by up to 50% compared to existing baseline policies.

1 Introduction

The rapid expansion of generative AI models, such as GPT [1], OPT [50], and LLaMA [45], has dramatically increased the computational and energy demands of AI training workloads [23, 24]. By 2030, data centers are projected to consume approximately 8% of total U.S. electricity. AI is the primary driver of this growth, with its share of data center power demand expected to increase from just 2.7% in 2023 to over 30% by year 2030 [36] (see Figure 1). Some analyses suggest this figure could be as high as 50-70% by then [27, 34]. This exponential increase in demand will soon reach a fundamental physical constraint: the limited capacity of regional power grids. While AI workloads are scaling rapidly, the underlying grid infrastructure remains stagnant, resulting in a situation where the power demand from a single data center can exceed the

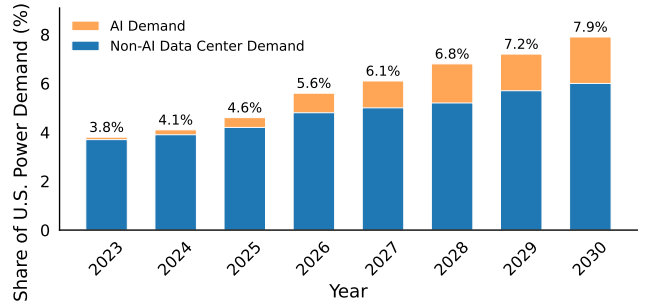


Figure 1: Projected growth of data center power demand [36].

ability of the local grid to reliably supply power to it [5, 32]. In addition to limited power availability, data centers may also have limited compute resources due to GPU shortages within a single region [48]. The limited availability of both power and compute resources can significantly degrade the performance of ML training in the data centers.

Power and computational resource constraints make the distribution of training workloads across multiple geographically distributed sites a necessity. Prior work has explored distributing ML training for large models across multiple geographical locations to leverage additional computational resources and improve training efficiency [16, 44, 49]. Decentralized [41, 43, 44] and federated [17, 38] training approaches enable the utilization of compute resources across geographically distributed data centers, mitigating GPU shortages at any single location [49].

While distributed large ML model training presents a promising solution to resource scarcity, it introduces a challenge: communication overhead. For instance, transmitting the 140GB LLaMA-2 model for an all-reduce operation over a 10 Gbps network can add two minutes of communication delay to every training iteration, which significantly increases the total training time.

Prior work has attempted to address this problem in two main ways. The first approach focuses on reducing the volume of data communicated. Systems like Photon [38] and DiLoCo [17] leverage techniques like low-rank adaptation and infrequent synchronization to reduce the communication cost of distribution. However, these methods typically operate on a *fixed*, pre-configured number of sites and do not adapt dynamically—as we show in Section 5.3, this results in significant opportunity losses. Further, these systems fail to consider the fact that different amounts of power are available at different sites.

Determining the optimal number of sites to use is not only resource-intensive but also highly dependent on the specific workload. Choosing too few sites negatively affects performance, while choosing too many creates a communication bottleneck. The second approach leverages heuristics for distribution, such as selecting all available resources within a fixed geographical radius to ensure low latency [22, 43]. The limitation of such a strategy is that it may ignore sites with more power that lie just outside the radius, and it fails to consider that a site within the radius might lack the power capacity to effectively use its hardware.

To address this problem, we introduce PowerTrip, a system designed to optimize the power-communication trade-off. The limitations above highlight a more fundamental optimization problem. Distributing training across more sites provides access to greater aggregate power, which improves computational speed and reduces time-to-accuracy. However, this distribution also increases communication costs, as model updates must be synchronized across a wider and more dispersed network. At a certain point, the performance penalty from this communication overhead outweighs the benefits of additional computational power. The core challenge is that this optimal balance is not static: it depends on the specific power capacities and network conditions of the available sites at any given time. Thus, PowerTrip’s primary goal is to maximize training performance by dynamically selecting the optimal number of sites at runtime. PowerTrip achieves this goal by addressing three fundamental and interconnected questions:

- ❶ **Which sites to choose?** PowerTrip ranks all available sites using a power-to-cost heuristic and selects sites that offer the best combination of high power availability (for better performance) and low communication cost.
- ❷ **How many sites to choose?** PowerTrip employs a dynamic greedy heuristic approach and uses the marginal gain in training efficiency (accuracy improvement per unit of time) to identify the optimal number of sites after which the performance penalty from communication overhead negates the benefit of adding more computational power.
- ❸ **How to scale the number of sites?** PowerTrip implements a greedy bottom-up scaling policy. It begins with a small set of high-power, low-latency sites and incrementally adds more if the marginal gain in training efficiency is positive. This approach avoids the significant initial communication costs of a top-down or fixed / static strategy and ensures that the communication cost per unit of accuracy gain remains low by design.

This paper presents our method for optimizing site selection to minimize training time by balancing the benefits of distributed power with the communication costs of model synchronization. To achieve this, PowerTrip’s dynamic optimization is twofold: at runtime, it decides not only *which* sites to select based on their individual efficiency gains, but also *how many* sites to include in the training process while minimizing the network delay. This second decision is important, as the quantity of selected sites determines the total aggregate power and, thus, the speed of computation.

Contributions. We summarize our main contributions as follows:

- We formulate and analyze an important trade-off between increased performance from distributed power availability and the

communication delays incurred when training large ML models across geographically distributed sites.

- We propose PowerTrip, a novel system that uses a dynamic greedy heuristic to select the optimal number of sites at runtime, maximizing training performance in power-constrained, distributed environments.
- We integrate publicly available Google data center power traces [37] into our experimental setup. This allows us to model and validate our system against real-world power constraints.
- We implement PowerTrip using the Flower framework [7] as an aggregator-based distributed training infrastructure and evaluate it across multiple datasets. Our results demonstrate that PowerTrip reduces the total training time by **44%** against an optimal static policy using uniform power profiles and by **25%** against the same static policy using Google power traces. Furthermore, compared to a power-constrained centralized approach, PowerTrip is more efficient by **50%** (with uniform power) and **27%** (using the Google trace power). This demonstrates that PowerTrip efficiently leverages distributed power while mitigating its network overhead.

2 Problem Formulation

This section details the power and compute constraints that motivate geo-distributed training, then formalizes the system architecture and its communication costs to define the power-communication trade-off, our system optimizes.

2.1 Power and resource constraints

Training state-of-the-art AI models is a highly resource-intensive process that has resulted in primary bottlenecks that restrict performance within a single data center: limited availability of power capacity and of high-performance hardware (e.g., GPUs).

Power capacity model. A data center, which we refer to as a site in this paper, is fundamentally constrained by its maximum power supply from a local electric grid. As the computational requirements for AI have grown exponentially [14] their power demand has begun to exceed the capacity of an individual site. We model a site’s computational throughput—its total processing power—as a direct function of its available power. Let a site k have a maximum power capacity of $P_{cap,k}$ (in watts). For instance, Google reports an average PUE of approximately 1.10, where infrastructure overhead like cooling accounts for only about 10% of the total power draw [20]. Our analysis therefore focuses on the computational power budget, as it is the primary driver of both energy consumption and performance limitations.

The site hosts a number of computational devices, such as GPUs, where each active GPU consumes an average power of P_{gpu} . The number of GPUs, $N_{gpu,k}$, that can be concurrently active at site k is therefore constrained by: $N_{gpu,k} \leq P_{cap,k}/P_{gpu}$. This inequality establishes a limitation on the amount of parallel computation a single site can support. The computational throughput of the site, C_k , is directly proportional to the number of active GPUs.

For a realistic model, we leverage publicly available **Google data center power traces** [37]. These traces provide fine-grained, five-minute granularity production power utilization data from large-scale compute cell (equivalent to a site), each containing

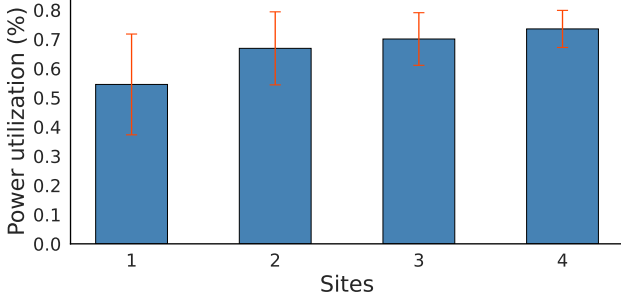


Figure 2: The power utilization at different sites in Google’s data center power traces.

roughly ten thousand machines. Figure 2 illustrates the power utilization across various sites. The height of each bar represents the mean power utilization, while the error bars depict the minimum to the maximum measured value for each respective cell. Across the 10 sites, the mean production power utilization varies from 49% to 74%, which shows that a site’s power utilization is highly dynamic. Additionally, Figure 3 shows that the power draw for any given PDU (power distribution unit for each rack of servers) at these sites is highly dynamic over time. This shows that a site may have a large inventory of physically available and idle hardware (e.g., CPUs or GPUs), but if the total power draw is already at its maximum, these resources cannot be fully utilized. This means that a site’s effective computational capacity is not determined by its hardware inventory alone, but by its *available power capacity* given by $P_{available,k}(t) = P_{cap,k} - P_{utilized,k}(t)$, at any given time t .

2.2 Geo-Distributed Training with a Central Aggregator

To leverage computational resources from geographically distributed locations, prior work has explored multiple distributed ML techniques. For instance, federated learning or all-reduced distributed ML synchronization techniques [17, 38, 43]. However, in this paper, we use a common architecture that involves a central aggregator coordinating a set of distributed worker sites. This model is structurally analogous to Federated Learning but is motivated by resource aggregation rather than data privacy. Additionally we adopt **data parallelism** in this work as it treats each site as a self-contained “worker” that can contribute to the overall training process. Each site holds a complete replica of the model and trains on its own partition of the data. This makes the system highly modular and scalable. Whereas the primary alternative, model parallelism, involves splitting the model itself across multiple sites, which creates tight dependencies between sites. In a dynamic, geo-distributed environment like ours, adding or removing a site would require a disruptive re-partitioning of the entire model mid-training, which is not feasible. Data parallelism avoids this completely.

Distributed training with aggregator. The global objective of this technique is to find the optimal model parameters θ^* that minimize a global loss function $L(\theta)$, defined as a weighted average of the local loss functions $L_k(\theta)$ at each of the K participating sites:

$$\theta^* = \arg \min_{\theta} L(\theta) \quad \text{where} \quad L(\theta) = \sum_{k=1}^K w_k L_k(\theta) \quad (1)$$

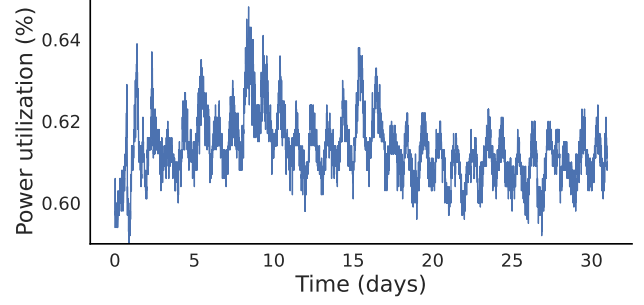


Figure 3: Time-series variation of power utilization for a single Power Distribution Unit (PDU) in one of Google’s data center.

Here, w_k is the weight for site k , typically proportional to the size of its local dataset D_k . Training proceeds in synchronous communication rounds, indexed by t . In each round, the process begins with a *broadcast* phase, where the aggregator sends the current global model parameters, θ^t , to a selected subset of S sites. Following the broadcast, each site k enters a *local computation* phase, using θ^t to initialize its local model and performing E epochs of training on its local data D_k to yield an updated local model, θ_k^{t+1} . Subsequently, during the *update communication* step, each site computes its local update, $\Delta_k^{t+1} = \theta_k^{t+1} - \theta^t$, and transmits it back to the aggregator. Finally, the round concludes with the *aggregation* phase, where after receiving updates from all S sites, the aggregator computes the new global parameters using the formula:

$$\theta^{t+1} = \theta^t + \sum_{k=1}^S w_k \Delta_k^{t+1} \quad (2)$$

Communication delays. While geo-distributed training presents a solution to resource limitation, it introduces the significant challenge of communication overhead. In a synchronous distributed system, the total time for a single training round (T_{round}) is the sum of the computation time ($T_{compute}$) and the communication time (T_{comm}). The communication time is dominated by the need to synchronize model parameters or updates across wide-area networks (WANs). For a model update of size (D_m) being sent from a site k to a central aggregator, the communication time is at least:

$$T_{k,comm} \geq \frac{2d_k}{c} + \frac{D_m}{B_k} \quad (3)$$

where d_k is the physical distance of site k to the aggregator, c is the speed of light in the propagation material, and B_k is the available bandwidth. In a synchronous round over a set of sites S , the system is bottlenecked by the slowest site (the “straggler”), making the round’s communication time $T_{comm}(S) = \max_{k \in S} \{T_{k,comm}\}$. The total communication time for the round is then determined by the maximum of these individual times over the set of selected sites S :

$$T_{comm}(S) = \max_{k \in S} \left\{ \frac{2d_k}{c} + \frac{D_m}{B_k} \right\} \quad (4)$$

Given that inter-continental latencies can be over 100ms and model updates can be gigabytes in size, this overhead can easily nullify the performance gains from more power availability, making the site selection optimization an important research problem.

2.3 The Performance Trade-off

The overall goal of our system is to reach a target accuracy in the least amount of time (i.e., minimize the **time-to-accuracy**) given the power constraints at a single site, greater aggregate power across multiple sites, and the communication overheads. This can also be framed as maximizing the training efficiency per unit of time. We define training efficiency, η , as the accuracy gain per second:

$$\eta(S) = \frac{\mathbb{E}[\text{Accuracy Gain per Round}]}{T_{\text{round}}(S)} \quad (5)$$

where the total round time for a set of sites S is a function of both computation and communication. The computation time depends on the effective throughput of the selected sites, which is constrained by both hardware availability and real-time power capacity. Following the model established from the Google power traces, the effective number of GPUs at a site k at time t , denoted $N_{\text{eff},k}(t)$, is the minimum of the physically available GPUs, $N_{\text{avail},k}$, and the number of GPUs that can be supported by the site's available power at that moment, $P_{\text{avail},k}(t)$. We formalize this as:

$$N_{\text{eff},k}(t) = \min \left(N_{\text{avail},k}, \left\lfloor \frac{P_{\text{avail},k}(t)}{P_{\text{gpu}}} \right\rfloor \right) \quad (6)$$

Therefore, the site's effective throughput at time t is directly proportional to this number: $C_k(t) = N_{\text{eff},k}(t) \cdot C_{\text{single_gpu}}$ where $C_{\text{single_gpu}}$ is the processing power of the single GPU. This model captures the scenario where a site's computational output is limited by its power budget, even if it has a surplus of idle hardware. Thus the total round time $T_{\text{round}}(S) = T_{\text{compute}}(S) + T_{\text{comm}}(S)$ is:

$$T_{\text{round}}(S) \propto \frac{1}{\sum_{k \in S} C_k(t)} + \max_{k \in S} \left\{ \frac{2d_k}{c} + \frac{D_m}{B_k} \right\} \quad (7)$$

This formulation makes the trade-off explicit: as we increase the number of sites $|S|$, we aggregate their independent power budgets. This increases the total effective computational throughput $\sum C_k(t)$ by allowing more of the available hardware to be utilized, which decreases the computation time, $T_{\text{compute}}(S)$, and in turn increases training efficiency. At the same time, as $|S|$ increases, the maximum communication time, is non-decreasing and likely to increase as more distant or lower-bandwidth network sites are included. This increases the total round time, decreasing training efficiency.

Due to these opposing effects, there exists an optimal subset of sites, S^* , that maximizes the training efficiency $\eta(S^*)$. Choosing too few sites leaves valuable computational power untapped, prolonging training. Choosing too many sites results in the system being bottlenecked by communication delays, again prolonging training. The challenge, therefore, is to dynamically identify this optimal set S^* at runtime. PowerTrip optimizes this tradeoff to maximize performance gains under power constrained environments.

3 PowerTrip Design

Figure 4 provides an overview of PowerTrip's design, which solves the power-communication trade-off by intelligently and dynamically selecting the optimal subset of available sites at runtime to maximize training performance. PowerTrip is designed to integrate with and expand upon existing distributed learning frameworks.

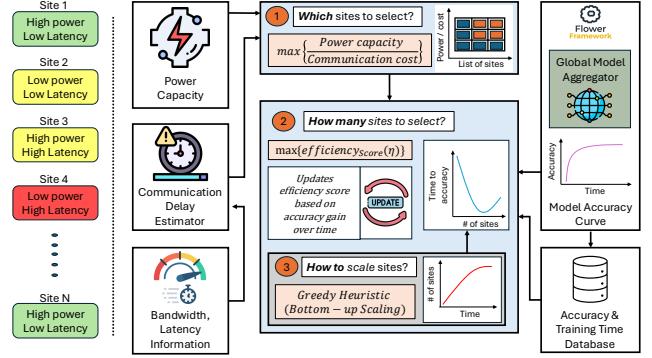


Figure 4: PowerTrip's system design and its components.

Existing frameworks [19, 35] already handle fundamental functions such as communication, site configuration, and model aggregation. However, these frameworks typically lack native support for profiling power availability and network delays (bandwidth and latencies) of distributed sites, which we enable by implementing the necessary auxiliary modules. As detailed in Section 4, we chose the Flower framework [7] for our implementation due to its flexible API, which allowed us to integrate our dynamic site-selection policy. While we use Flower, our design is framework-agnostic and can be applied to any system that allows for the configuration of selected sites in each synchronous training round.

We next detail PowerTrip's modules and describe how they enable it to make three core decisions that optimize the power-communication trade-off:

- (1) Which sites to select to maximize performance by exploiting high power availability and low communication cost? (Section 3.1)
- (2) How many sites to choose to identify the optimal number of sites, after which the performance penalty from communication overhead negates the benefit of more power availability? (Section 3.2)
- (3) How to scale the number of sites to ensure that the communication cost per unit of accuracy gain remains low? (Section 3.3)

In presenting these modules, we highlight the design decisions that enable PowerTrip to dynamically optimize heterogeneous power-communication trade-offs.

3.1 Site selection for performance gains

PowerTrip's site selection strategy aims to rank all available sites using a power-to-cost heuristic and then selects sites that offer the best combination of high power availability (for better performance) and low communication costs. This strategy draws inspiration from two simpler heuristics: the lowest network latency heuristic and the highest power heuristic, which serve as baselines.

A site $k \in S$ is characterized by its effective power availability, $P_{\text{avail},k}(t)$, and its communication delay to the aggregator, $T_{k,\text{comm}}$. A selection strategy can prioritize one or both of these metrics:

Lowest Latency. This strategy prioritizes minimizing communication overhead, $T_{k,\text{comm}}$ as given in Equation 3, by creating a ranked list of candidates based on their individual communication delays. The final output of this strategy is an ordered list of all available sites, with those exhibiting the best connectivity (lowest latency)

positioned at the top. It is designed for scenarios where the overall training time is dominated by communication bottlenecks, such as in geographically dispersed or bandwidth-constrained environments where fast synchronization is important. However, its primary weakness is that it does not explicitly consider computational power, $P_{avail,k}$ and as we established in Section 2.1, power availability varies significantly across sites. Consequently, this policy will perform poorly if the sites with the lowest communication cost are also the ones with the lowest power capacity, as the training would then be limited by slow computation.

Highest Power. This strategy prioritizes maximizing power availability by ranking all available sites $k \in \mathcal{S}$ solely based on their effective computational power, $P_{avail,k}$. The resulting output is an ordered list of the sites with most power available. This approach is designed for scenarios where local computation is the primary bottleneck, such as when training large scale models. Because this strategy does not explicitly consider network latency, it is most suitable when all sites have high-bandwidth, low-latency connections, making communication time a negligible factor in the overall training duration. However, as discussed in Section 2.3 the communication delays can be significant in a distributed training environment, thus this policy is likely to select a powerful data center that is geographically distant, which then becomes a high-latency “straggler” in every synchronous training round. This forces all other faster sites to wait and delays the training time.

PowerTrip’s selection strategy. Given that strategies focusing on a single metric are prone to creating computational or communication bottlenecks, PowerTrip’s selection strategy provides a balanced approach that simultaneously considers both power availability and the communication delays. However, we recognize that the relative importance of computational power versus communication cost is not static; it can change depending on the size of the model update being transmitted, given that bandwidth remains the same. To account for this, our strategy uses an adaptive, weighted scoring mechanism.

The idea is to calculate a weighted power-to-cost score for each site $k \in \mathcal{S}$. This score is designed to be flexible, adjusting its priorities based on the characteristics of the training workload. The score is calculated as:

$$\text{Power-to-cost score}(s_i) = \frac{P_{avail,k} \cdot w_p}{T_{k,comm} \cdot w_c} \quad (8)$$

where w_p and w_c are dynamic weights that control the emphasis on power versus communication cost. These weights are adjusted based on the model update size, D_m , to adapt the selection heuristic to the workload. For small update sizes, where overall training time is more sensitive to local computation speed, the system prioritizes power by setting $w_p > w_c$. Conversely, for large updates sizes, where network latency becomes the dominant bottleneck, it prioritizes low communication cost by setting $w_c > w_p$. This allows the ranking to effectively identify the most valuable sites for different training scenarios.

3.2 Site provisioning and power-cost trade-off

Once a site selection strategy generates a ranked list of candidates with the best power-to-cost score, the next design question is to determine the optimal number of sites to select during the training

to balance the benefit of more computational power against the cost of higher communication delays. In this section, we explore different policies for making this decision, which serve as baselines for PowerTrip’s dynamic policy, which we discuss in Section 3.3.

Select all sites. This strategy represents a simple baseline where the system utilizes every available site for training ($\mathcal{S}' = \mathcal{S}$), assuming that maximizing parallel computation also yields maximum performance. However, this approach fails to consider the impact of communication overhead in a synchronous environment.

As shown in our round time model in Equation 7, while selecting all sites maximizes the aggregate power (P_{avail}) and minimizes the computational time $T_{compute}(\mathcal{S})$, it also maximizes the communication time $T_{comm}(\mathcal{S})$, which is dictated by the site with the highest latency in the entire set of selected sites ($\max_{k \in \mathcal{S}} T_{comm,k}(\mathcal{S})$). In any large-scale, geographically distributed system, this value creates a synchronization bottleneck, forcing all high-power, low-latency sites to idly wait. As a result, the training time for each round becomes dominated by communication, which reduces overall performance.

This strategy serves as a useful baseline, demonstrating that blindly maximizing computational resources is an ineffective approach and motivates a more intelligent selection mechanism.

Static selection. This policy is an offline, analytical approach designed to empirically find the theoretical optimal *fixed* number of sites for a given workload. This policy directly explores the power-communication trade-off by running a series of exhaustive, end-to-end training experiments with the primary objective of finding the number of sites, k , that minimizes the total time required to reach a target model accuracy (T_{target_acc}). This time-to-accuracy is a product of the number of synchronization rounds needed (R_{acc}) and the average time per round (T_{round}), given by: $T_{target_acc}(S_k) = R_{acc}(S_k) \cdot T_{round}(S_k)$

An optimal number of sites, k^* , exists due to the competing effects on this time-to-accuracy as k increases. As more sites are added to the training group (S_k), the aggregate computational power ($\sum P_{avail,k}$) increases, which reduces the computation time per round. In contrast, including more sites (which are selected from progressively farther away) increases the likelihood of high-latency sites, which increases the communication time per round ($\max(T_{comm}(S))$). This creates a distinct trade-off where the initial benefits of adding more power are eventually outweighed by the communication costs, ensuring that a minimum value for T_{target_acc} exists at some optimal k^* .

This creates a distinct trade-off. Initially, for small k , adding sites dramatically reduces training time because the computation gains are significant and outweigh the minor increases in latency. However, after a certain point, there are diminishing returns. The added sites are farther away, contributing less marginal computational power while significantly increasing the communication bottleneck. This causes the round time, and consequently the total training time, to increase. This behavior ensures that a minimum value for T_{total_acc} exists at some optimal k^* .

This approach for finding the optimum is straightforward but resource-intensive. For a chosen site selection strategy, we first generate a ranked list of all available sites. We then execute the full training process for various group sizes, $k \in \{k_1, k_2, \dots, S\}$ sites. For each complete run, we measure the final time-to-accuracy. The

Table 1: Variables in PowerTrip’s dynamic policy

Variable	Description
k	A candidate number of sites to evaluate
k_{current}	The number of sites used in the current round
$\mathcal{K}$	The set of candidate counts for the next phase
α_r	Model accuracy at the end of round r
$\Delta\alpha_k$	Projected accuracy gain for using k sites
τ_k	Estimated round time for using k sites
η_k	Training efficiency score for using k sites
α_{target}	Target accuracy threshold to stop training
p	Patience: number of rounds to wait before reducing sites

optimal fixed number of sites, k^* , is then identified as the value of k that resulted in the lowest total training time:

$$k^* = \arg \min_k T_{\text{target_acc}}(S_k) \quad (9)$$

The primary drawback of this policy is that it is an *offline, post-hoc analysis*, making it too resource-intensive. However, it provides an important baseline against which we evaluate PowerTrip’s dynamic policy, as the resulting k^* represents the best performance achievable with any *fixed set of sites*.

3.3 PowerTrip’s Dynamic Optimization Policy

The dynamic optimization policy is the core of our proposed system, PowerTrip. Unlike the fixed Static Policy, this approach determines the optimal number of training sites at runtime, continuously adapting to the real-time power and network conditions of the distributed environment. Our goal is to actively manage the power-communication trade-off during training to minimize the overall time-to-accuracy.

Greedy heuristic for runtime optimization As detailed in Algorithm 1, the training process begins by initializing the active site count to a small number, $k_{\text{current}} = \min(k_{\text{current}}, S)$, where S is the total number of available sites. These sites are the best-ranked sites, i.e., those with the highest Power-to-Cost Score as discussed in Section 3.1. After the first training round is complete, the system must decide how to scale the number of participants for the next round. To address this, our system adopts a **bottom-up, greedy incremental addition strategy**. This approach starts with a small, core group of the best-ranked sites, i.e., those with the highest Power-to-Cost Score. It then greedily adds more sites in subsequent rounds. This strategy is more efficient than a top-down, subtractive method that begins with all available sites as the initial training rounds would be slow due to the communication bottleneck from including all high-latency sites.

The system then evaluates a set of potential site counts, $\mathcal{K}$, within a window $k_{\text{current}} \pm n$ around its current count k_{current} . For each candidate count $k \in \mathcal{K}$, the system projects the expected accuracy gain, $\Delta\alpha_k$, using a model that incorporates recent accuracy trends ($\alpha_r - \alpha_{r-1}$), a normalization factor k/k_{current} for the number of workers, and a logarithmic term to account for diminishing returns.

Concurrently, the system estimates the round time τ_k as the sum of two components: the computation time, which is inversely proportional to the mean power capacity (or computational capability) of the selected sites, and the communication time, which is determined by the maximum network latency among the selected sites. Specifically, $\tau_k = \frac{T_{\text{comp}}}{\text{mean}(P_{\text{avail}}(1:k))} + \max(T_{\text{comm}}(1:k))$, where

Algorithm 1: Greedy Heuristic Dynamic Site Selection

Input : S sites ranked by power-cost-score $s_i = \frac{P_{\text{avail}}}{T_{\text{comm}}}$
 (P_{avail} : available power, T_{comm} : network latency)

Output: Trained model parameters after reaching α_{target}

```

1 Initialization:
2  $k_{\text{current}} \leftarrow \min(\text{initial\_sites}, S)$ 
3  $r \leftarrow 0$ ,  $\text{last\_improvement} \leftarrow 0$ 
4 while  $r \leq r_{\text{max}}$  and  $\alpha_r < \alpha_{\text{target}}$  do
5   Training Round:
6   Select top  $k_{\text{current}}$  sites from ranked list
7    $\alpha_r, \tau_r \leftarrow \text{TrainModel}(k_{\text{current}})$ 
8   if  $r - \text{last\_improvement} > p$  then
9     break
10    // Stop if accuracy has plateaued
11  if  $r \bmod \text{adjustment\_interval} = 0$  then
12     $\mathcal{K} \leftarrow \text{GetCandidateCounts}(k_{\text{current}}, K)$ 
13    // e.g.,  $k_{\text{current}} \pm n_1, \pm n_2$ 
14    for each  $k \in \mathcal{K}$  do
15       $\tau_k \leftarrow \frac{T_{\text{comp}}}{\text{mean}(P_{\text{avail}}(1:w))} + \max(T_{\text{comp}}(1:k))$ 
16       $\Delta\alpha_k \leftarrow$ 
17        ProjectAccuracyGain( $k, k_{\text{current}}, \alpha_r, \alpha_{r-1}$ )
18      // See text for formula
19       $\eta_k \leftarrow \frac{\Delta\alpha_k}{\tau_k}$ 
20       $k^* \leftarrow \arg\max_{k \in \mathcal{K}} \eta_k$ 
21      if  $k^* < k_{\text{current}}$  and  $r - \text{last\_improvement} < p$  then
22         $k^* \leftarrow k_{\text{current}}$ 
23        // Avoid premature reduction
24       $k_{\text{current}} \leftarrow k^*$ 
25  if  $\alpha_r > \alpha_{r-1}$  then
26     $\text{last\_improvement} \leftarrow r$ 
27   $r \leftarrow r + 1$ 

```

P_{avail} represents the power capacity at a single location and T_{comm} represents the network latency for the top k sites.

At each synchronization round, the optimal site count k' is then selected by maximizing the accuracy-efficiency gain $\eta_k = \frac{\Delta\alpha_k}{\tau_k}$ across all candidates $k \in \mathcal{K}$. This ratio effectively balances accuracy improvements against time costs, ensuring the system selects the site count k' that delivers the best accuracy-per-time-unit performance until the system reaches the target accuracy.

Advantage in time-to-accuracy. The primary advantage of our dynamic policy over the static policy lies in its better efficiency over the entire training life-cycle. A static policy using a large, fixed number of sites, k^* , is often bottlenecked by the high communication cost of its slowest site in every single round. The total training time is the sum of these consistently slow rounds: $T_{\text{static}} = \sum_{r=1}^{R_{\text{static}}} T_{\text{round}}(S_{k^*})$.

In contrast, our dynamic policy begins with a small, highly efficient group of sites, k_0 , where the maximum latency is minimal, making the initial training rounds faster, with relative accuracy gain compared to the k^* . The policy then intelligently scales the number

of sites, k_r , based on its runtime efficiency heuristic. The total time is thus a sum of varying round times, many of which are much faster than the static policy’s rounds: $T_{\text{dynamic}} = \sum_{r=1}^{R_{\text{dynamic}}} T_{\text{round}}(S'_{k_r})$. By starting small and scaling up when efficient, the dynamic policy reduces training time in the early and middle stages of training.

4 Implementation

We implement PowerTrip using Flower [7], a widely-used open-source framework for federated and distributed learning, which enables us to focus on PowerTrip’s core logic rather than its underlying communication protocols. Our implementation follows a centralized architecture composed of a single aggregator, which hosts the core system logic, and multiple distributed sites, each representing a training site. The aggregator organizes the entire training process, including profiling the client sites, executing the dynamic site selection algorithm, and aggregating model updates. Sites receive the global model, train locally, and report model updates and status to the aggregator. PowerTrip supports QLoRA [15] or LoRA [21] optimizations for training large language models, reducing network overhead by reducing update parameter sizes.

PowerTrip is composed of several key modules. A *Site Profiler module*, running on the aggregator, gathers and maintains the information about each available site. For our experiments, we model two key characteristics for each site k , using both controlled configurations and real-world data traces. For controlled experiments, the computational capacity, C_k , is a pre-configured parameter set at the start to simulate sites with different maximum power capacities. To emulate real-world power capacities, we also leverage Google’s publicly available power traces. From these traces, our Site Profiler computes the spare power capacity available for computation at any given time. This allows our system to model a realistic scenario where a site may have sufficient idle compute resources, but its performance is ultimately limited by the lack of available power. The profiler is designed to handle this discrepancy, ensuring that site selection is based on actual power availability, not just hardware inventory. In both scenarios, the network characteristics, L_k and B_k , are managed using network emulation tools to control the latency and bandwidth between each client site and the aggregator. The Site Profiler periodically measures these network values to dynamically update the communication cost model used by the selection logic.

The *Site selection module*, which is the core of our contribution, implements our dynamic greedy heuristic. At the start of each selection phase, it ranks all available sites based on a performance-to-cost heuristic that prioritizes high computational throughput and low communication cost. The module then builds the optimal set of sites incrementally. Starting with the top-ranked site, it iteratively considers adding the next-best site from the list. A site is only added if the system estimates that its inclusion will improve the overall training efficiency compared to the previously measured performance. This process continues until adding more sites offers no further benefit, yielding the optimal set for the next training phase. Finally, for training execution and monitoring, the aggregator uses standard mechanisms to dispatch tasks to the clients selected by the Site Selection Module. We instrumented the aggregator to measure the wall-clock time for each training round and

to evaluate the global model’s accuracy on a held-out validation set after each round. This allows the system to compute the realized training efficiency, η_{measured} (Equation 5), which, as discussed in Section 3, serves as the feedback signal for the selection module in the subsequent decision-making cycle.

5 Evaluation

Below, we present our evaluation methodology (Section 5.1), evaluate PowerTrip’s design components (Section 5.2 and Section 5.3), and analyze PowerTrip’s robustness (Section 5.4).

5.1 Evaluation Methodology

We present our evaluation methodology, covering (1) compute resources, (2) training datasets and models, (3) baseline policies, (4) evaluation metrics and (5) power and communication profiling.

Table 2: Statistics for datasets used in the evaluation.

Dataset	Task	Model	Classes	Samples
EMNIST	Image Class.	CNN	62	814,255
Shakespeare	Next-Char Pred.	LSTM	80	422,615
Google Speech	Keyword Spot.	KWT-1	35	105,829

Experimental setup. We evaluate PowerTrip on a local cluster of 40 NVIDIA GTX 1080Ti GPUs. We emulate a distributed machine learning environment consisting of 100 unique sites, each with distinct training and communication delays. The underlying communication protocols and multi-node orchestration are provided and managed by the Flower framework [7].

Simulated time and resource emulation. Given that the number of emulated sites exceeds our available physical GPUs, we employ a simulation strategy to model performance under perfect parallelism. In our setup, a single physical GPU may be responsible for executing the training tasks for multiple virtual sites. These tasks run sequentially on the hardware; however, we calculate a *simulated wall-clock time* for each round to accurately reflect how the system would perform if each site had its own dedicated resources running in parallel. This environment also allows us to simulate and evaluate different parameter update sizes which affect the communication time. The total time for a site k is the sum of its computation time and communication time. The communication time is described in Equation 3 whereas the computation time is determined by scaling the emulated time by the site’s emulated power availability ($P_{\text{avail},k}$). This method allows us to evaluate our policy’s ability to handle resource heterogeneity at a scale that would otherwise be infeasible, while accurately modeling the performance bottlenecks that arise in a real parallel execution environment.

Datasets and models. We evaluate our approach on three datasets representing distinct tasks and complexities. For the EMNIST image recognition dataset [11], we train a simple Convolutional Neural Network (CNN) from scratch with a learning rate of 0.01. For next-character prediction on the Shakespeare text corpus [9], we use a two-layer LSTM [28] with a learning rate of 0.8. Finally, for speech classification on the Google Speech Commands dataset [46], we train a KWT-1 model [6] with a learning rate of 0.001. The batch size was 20 for EMNIST and Shakespeare, and 16 for Google Speech. The characteristics of each dataset are summarized in Table 2.

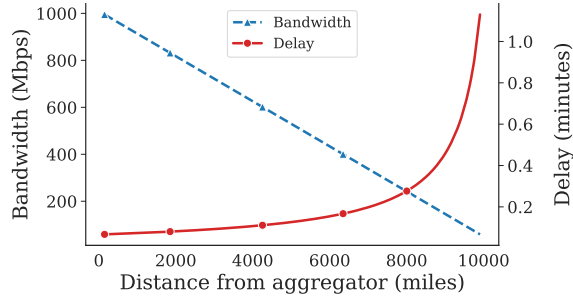


Figure 5: Bandwidth decreases where as communication delay increases with increasing geographical distances.

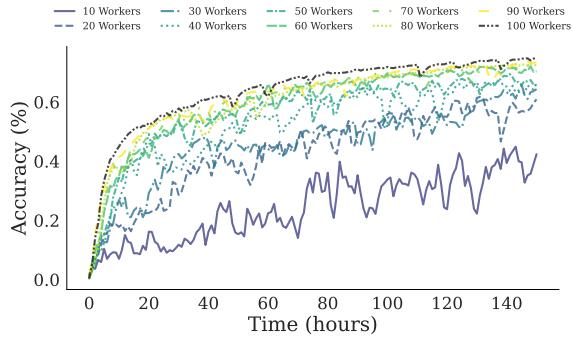


Figure 6: Using more sites improves convergence accuracy due to the greater aggregate computational power available, assuming the same power and network delay across all sites.

Baseline policies. Below, we present the baseline approaches that we compare PowerTrip with.

- **Random selection.** As a baseline for comparison, we use *Random Selection*, a common approach in distributed learning [8, 47]. In this method, a random subset of available sites is chosen to participate in each training round. This randomness is a primary weakness in a performance-oriented system. Random selection is agnostic to the resource profiles of the sites, meaning it might select a site with low computational throughput due to power limitations or a site with high network latency, impacting the performance negatively in both cases.
- **Power constrained centralized training.** In this scenario, we emulate the training process occurring at a single site. The primary advantage of this approach is no network communication overhead, as all computation happens at one location. The drawback, however, is that performance is strictly limited by the power capacity of that single site. Even if more hardware is physically available, it cannot be utilized once the site’s power cap is reached, thus capping the maximum computational throughput [5, 33]. This baseline is important as it quantifies the performance capacity for any non-distributed approach and highlights the motivation for geo-distribution: accessing more aggregate power, while incurring communication costs.
- **Fixed radius.** Our second baseline is *Fixed Radius Selection*, which selects all available sites within a predefined geographical distance. The primary advantage of this strategy is low communication latency due to the proximity of sites, leading to faster

synchronization rounds. Its drawback, however, is being constrained by the power resources available within that local area, as it may ignore more sites with more power that are farther away. This can result in a system that is bottlenecked by computation despite its efficient communication. This approach is motivated by prior work [43] that suggests keeping data parallelism within a single region to avoid high communication costs. However, that work does not analyze the potential performance gains from accessing greater aggregate power at the cost of higher latency, making this an essential baseline for our study.

- **Static Selection.** This baseline represents a static approach where a fixed number of sites, k , is selected at the start of training and used for every subsequent round. This fixed number of site selection is observed in prior work [17, 38] as well. The drawback of the fixed-static policy is that determining the optimal number of sites to select is not only resource-intensive but also highly dependent on the specific workload. Choosing too few sites negatively affects the performance, while choosing too many creates a communication bottleneck. However, it provides an important baseline against which we evaluate PowerTrip’s dynamic policy, as the set of selected sites with this policy represents the best performance for a *fixed set of sites*.

Power profiling. We model power constraints using two different methods. For controlled, repeatable experiments, we assign each site a static power availability value, drawn from a uniform random distribution between 0.1 and 1.0. This value represents the fraction of a site’s maximum potential computational throughput that is currently available. We then simulate the impact of this power constraint on the local training time. For instance, if a site’s ideal training time (at 100% power) is T_{ideal} and its power availability is P_{avail} , its effective training time becomes T_{ideal}/P_{avail} . This means a site with 50% power availability ($P_{avail} = 0.5$) would take twice as long to complete its local computation, allowing us to test our policies under specific, heterogeneous conditions. For dynamic, real-world scenarios, we use Google’s publicly available cluster power traces [37] to determine the spare power capacity for each site at any given time. This allows us to simulate realistic power fluctuations where performance is limited by the grid.

Communication delays. We synthetically generate the network profiles for our 100 emulated sites where each site is assigned a random geographical distance from the aggregator, ranging from 100 to 10,000 miles (distances). We model the inverse relationship between a site’s network bandwidth (bandwidths), which is set to be inversely proportional to its distance, visualized by left y-axis in Figure 5. The total communication delay for each site is then calculated using the Equation 3. For the given experiments, we also configure the parameter update size to be 1.5GB to emulate the communication delays associated with large scale ML training.

The right y-axis in Figure 5 illustrates the result of this model, showing how the total communication delay increases non-linearly with distance due to the combined effects of propagation time and decreasing bandwidth.

Evaluation Metric. The primary metric for our evaluation is *time-to-accuracy*. This metric measures the total wall-clock time required for a given training configuration to reach a predefined target accuracy on a held-out validation set. We choose this metric because

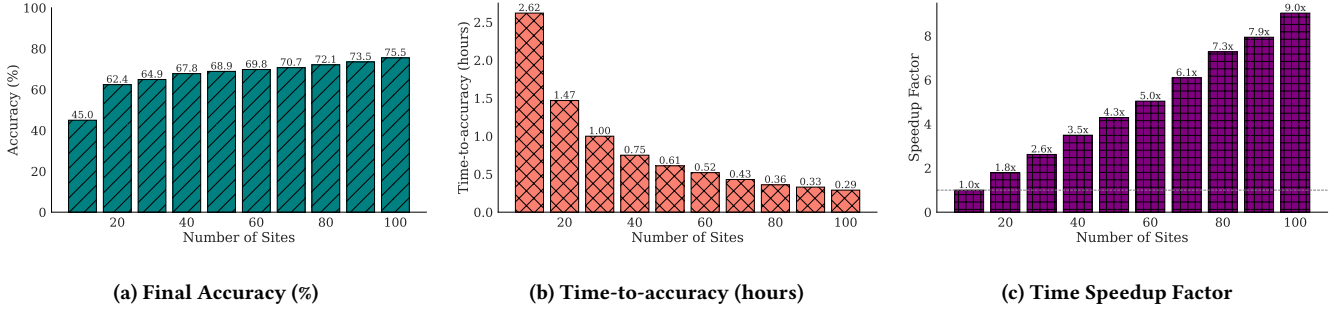


Figure 7: Performance metrics - (a) convergence accuracy, (b) time-to-accuracy and (c) speedup factor, improve with more sites, due to greater aggregate computational power available.

it provides a holistic measure of end-to-end system performance, capturing the impact of both computational throughput and network overhead. A lower time-to-accuracy indicates a more efficient system. We report results across 10 runs of each experiment.

5.2 Exploiting heterogeneous power availability

Below, we empirically validate the existence of the power-cost trade-off and show the existence of an optimal set of sites which exploit this trade-off to minimize the time-to-accuracy.

Performance gains with more power availability. (Figure 6) illustrates the direct impact of aggregate computational power on training performance in an idealized setting where all sites have identical power availability and communication costs, thus the communication cost for adding an additional site into the training set is essentially zero. The results clearly show that as more sites are added, the training process yields multiple benefits: the model converges to a higher final accuracy (Figure 7a), the time-to-accuracy (Figure 7b) is substantially reduced, and the overall training speedup (Figure 7c) increases significantly compared to the 10-site baseline. This experiment confirms our foundational premise that, *increasing the total aggregate power by adding more computational units directly improves overall training efficiency* in the ideal case where all the sites reside at a single location thus we can have additional power available without any communication delays. Although, we have zero communication cost for adding an additional power site, the returns of having more power available diminish over time in terms of time-to-accuracy.

Next, we introduce the communication delays based on the communication model discussed previously and discuss the trade-off between power and communication. Here, we observe that the effect of diminishing advantage of adding more sites and increasing communication costs over distances (with lower bandwidths) where selecting more sites into the training set costs more in terms of communication delays, as we discussed in Section 3.2.

Key point. *We show that in the absence of communication costs, maximizing aggregate power is always beneficial, though with diminishing returns.*

Power-communication trade-off. Before evaluating PowerTrip’s performance, we first empirically validate the existence of the power-communication trade-off using the static baseline. For this purpose, we first generate a ranked list of all 100 available sites based on our power-to-cost heuristic,

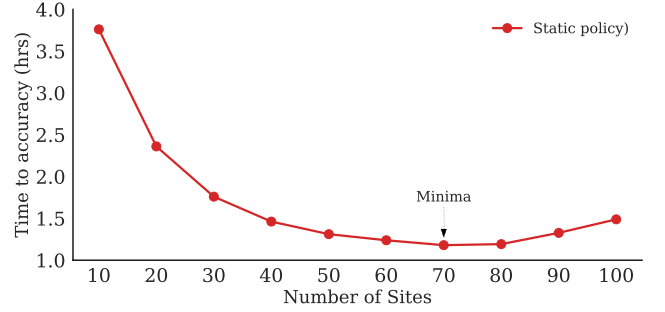


Figure 8: The static post-hoc policy identifies an optimal number of sites (in this case, $k = 70$) that minimizes training time, empirically validating the power-communication trade-off.

which prioritizes sites with high power availability and low communication delay. We then execute a series of complete, end-to-end training runs on the EMNIST dataset. Each run uses a fixed subset of sites from the top of this list, for instance, using the top $k \in \{10, 20, 30, \dots, 100\}$ sites, the x-axis in Figure 8. For each value of k , we measure the time-to-accuracy where the target accuracy is 45% in this case, which is the lowest achievable accuracy among all sites.

Figure 8 explores the trade-off between increased power availability and the communication delays. As we increase the number of sites, k , the total time-to-accuracy follows a convex curve. For a small number of sites k , adding more sites increases the aggregate computational power, resulting in a reduction in the computational time per synchronization round due to increased power availability far outweighing the increase in communication delay, which leads to a drop in the overall time-to-accuracy. However, as we continue to add sites with relatively low power availability and higher communication delays, the marginal computational gain from each new site decreases. Simultaneously, the probability of including a geographically distant site with high network latency increases. In Figure 8, the dashed red line shows that the optimal number of sites occurs at $k^* = 70$. After this, the penalty from the increased communication time begins to dominate the diminishing computational benefits, increasing the total time-to-accuracy. Importantly, the minima at k^* , is only valid for this specific configuration of the workload and network conditions. Any change would require another expensive, exhaustive search, highlighting the impracticality of a static approach and motivating the need for a robust, dynamic policy that can optimize k^* at runtime.

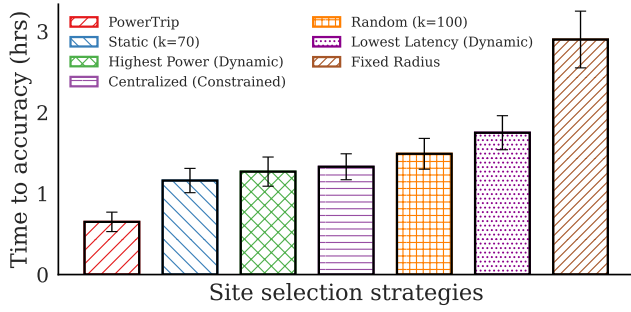


Figure 9: Performance evaluation of PowerTrip’s dynamic policy against the baselines.

The existence of this valley in the performance curve empirically shows that an optimal number of sites exists for a given configuration. While this offline method is too resource-intensive for practical use in a dynamic system, it provides a good benchmark. The performance achieved with k^* sites represents the best outcome for any policy that uses a fixed set of participants, and we use this result as a baseline against which we evaluate PowerTrip’s performance.

Key point. We use a static policy to empirically validate the power-communication trade-off, showing that performance degrades after an optimal number of sites is reached because communication costs begin to outweigh computational gains.

5.3 PowerTrip’s Dynamic Policy Performance

Having empirically validated the power-communication trade-off and identified the static configuration ($k^* = 70$), we now evaluate the performance of our dynamic greedy heuristic policy. As detailed in Section 3.3, PowerTrip is designed to find this optimal operating point at runtime, without the need for the exhaustive offline analysis required by the Static Optimal policy.

To demonstrate this, we conduct an end-to-end training experiment on the EMNIST dataset, comparing the time-to-accuracy of PowerTrip against the established baselines, including the static optimal policy fixed at $k = 70$ and a power constrained centralized training approach. This evaluation has two primary objectives. First, by comparing against the Static Optimal policy, we aim to show that our dynamic approach can achieve better performance without requiring any expensive, offline profiling to pre-configure the system. Second, by comparing against the centralized baseline, we demonstrate that the performance benefit gained from dynamically aggregating heterogeneous power across distributed sites can outweigh the inherent communication overhead. By monitoring the number of sites PowerTrip chooses in each round, we can observe its ability to optimize for the number of sites and balance the power-communication trade-off in a practical, online manner.

Figure 9 presents the results of our evaluation, comparing the time-to-accuracy of each policy. The result clearly demonstrates that PowerTrip outperforms all other strategies, achieving the target accuracy in just **0.65 hours**. The Fixed Radius policy, with a defined radius of 1000 miles (we assume this number based on our communication profiling Figure 5 which shows the bandwidth remains relatively constant in this radius), limited to only 22 nearby sites,

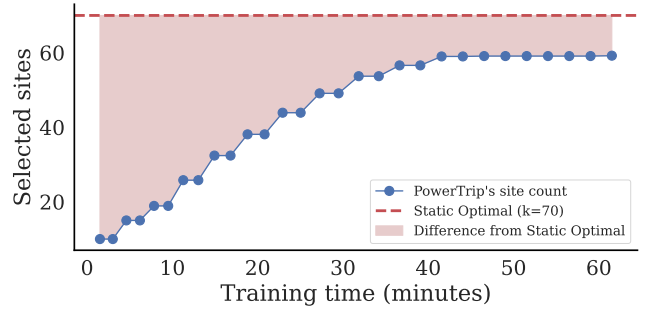


Figure 10: Number of sites selected by PowerTrip’s dynamic policy, which scales up over time to avoid high initial costs, against the fixed Static Optimal baseline.

is the slowest at 2.9 hours, as it is significantly bottlenecked by its limited access to aggregate power despite its low communication latency. Conversely, the policy that dynamically selects the Highest Power sites is also inefficient (1.27 hours), as it ignores communication costs and only includes powerful but distant sites that have high communication delays. Finally, Random selection performs poorly (1.49 hours) due to its inability to make informed decisions about site quality.

Most importantly, PowerTrip’s time of 0.65 hours is nearly **44% faster** than the Static Optimal ($k=70$) policy, which took 1.16 hours and is **50% faster** than the centralized approach with 25% power availability policy, which took 1.33 hours whereas it performs similar if the power constrained is 50%. This demonstrates the significant benefit of dynamic adaptation over a fixed, pre-computed strategy and a power-limited centralized approach.

Advantage in time-to-accuracy. Figure 10 provides a clear visualization of the operational difference between PowerTrip’s dynamic policy and the Static Optimal baseline. The figure plots the number of sites selected by each policy against the total elapsed training time. The Static Optimal policy, by definition, uses a fixed set of $k = 70$ sites from the very first round, as indicated by the horizontal dashed line. In contrast, PowerTrip’s policy employs a bottom-up scaling approach. It begins the training process with a small number of highest power, lowest communication latency sites—those offering the best ratio of computational power to communication cost. As the training progresses, it dynamically adds more sites as explained in Section 3.

The shaded area between the two curves represents the significant “site overhead” incurred by the static policy in the early stages of training. By starting with 70 sites, the static approach is immediately bottlenecked by the highest-latency site within that large group, making every single round slower. PowerTrip’s dynamic policy avoids this penalty. This cumulative time saved during the initial and middle phases of training is the primary reason for its performance improvement over static selection. To further validate the design of our greedy heuristic, we analyze the marginal benefit of adding more sites in terms of accuracy gain per round. Figure 11 plots the accuracy gain between larger site configurations (50 and 100 sites) and a small baseline of 10 sites. The key observation is that in the initial phase of training, e.g., the first 10–20 rounds, this difference in accuracy gain is small. While adding more sites does

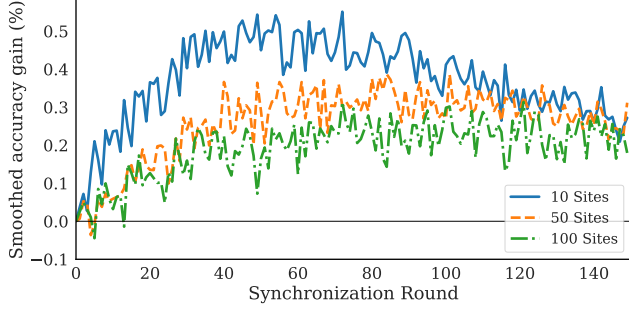


Figure 11: Accuracy gain per synchronization round.

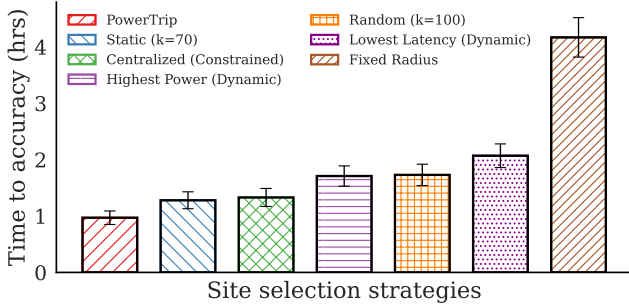


Figure 12: Performance evaluation of PowerTrip's dynamic policy against the baselines using the Google's data center production power utilization traces.

eventually lead to faster convergence, the immediate, per-round improvement in learning is small at the beginning.

This explains the fundamental inefficiency of a static policy. A static approach that starts with a large number of sites (e.g., $k = 70$) pays a large and fixed communication cost in every single round, all for a minimal increase in accuracy gain during the early stages of learning. In contrast, a bottom-up greedy approach starts with a small group of efficient, high-power, low-latency sites. In these initial rounds, it achieves the same relative accuracy progress as a large configuration but with significantly lower communication time. The cumulative time saved by avoiding this high, upfront communication tax is the primary reason PowerTrip achieves a better overall time-to-accuracy, even though it may scale up to a similar number of sites later in the training process. This validates that our online, greedy heuristic effectively navigates the power-communication trade-off by avoiding the over-provisioning of resources when the marginal benefit is low, leading to better performance compared to a static policy.

Evaluation with Google Power Traces. To evaluate the performance of our system under more realistic conditions, we use Google's publicly available data center power utilization traces [37]. In this scenario, the power availability at each site is not a synthetic value but is instead determined by the real-time spare capacity derived from the production workload traces. This allows us to model a dynamic environment where a site's computational throughput is limited by its power budget, not just its hardware inventory.

Figure 12 show that the relative performance trends among the policies are consistent with our controlled experiments (Figure 9):

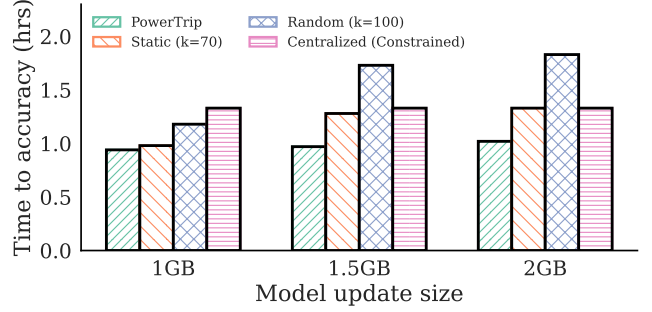


Figure 13: Time-to-accuracy comparison across varying model update sizes, demonstrating that PowerTrip consistently outperforms all baseline policies and highlighting the robustness.

PowerTrip remains the most efficient policy, outperforming all baselines. PowerTrip's dynamic policy reaches the target accuracy 24% faster than the optimal static policy and 27% faster than the centralized approach with 25% of power availability (which is the lowest power availability from the google power traces). However, a key insight is that the maximum available spare power at any given site in these traces is much lower than in our synthetic distribution, typically falling within the range of 26% to 51% of the total capacity. Because the overall power available to the system is more constrained, the absolute time-to-accuracy for *all* policies increases compared to the previous experiments. This result validates the robustness of PowerTrip's dynamic approach with the realistic power capacities but also emphasizes the influence of real-world power constraints on overall training performance.

Key point. PowerTrip outperforms existing baselines by leveraging dynamic scaling to optimize the number of sites at runtime that effectively exploit the power-cost trade-off to minimize time-to-accuracy.

5.4 Generalizability and Robustness

To ensure that PowerTrip's performance benefits are not limited to a single workload, we conduct further experiments to evaluate its generalizability across different datasets and its robustness to varying model update sizes.

Effect of varying parameter update sizes. To validate the robustness of our adaptive selection strategy, we conduct experiments using three distinct model's parameter update (that needs to be communicated between the aggregator and the sites) sizes: Small (1GB), Medium (1.5GB), and Large (2GB). As detailed in our design (Section 3.1), PowerTrip's site selection score is weighted to adaptively prioritize either computational power or low communication cost based on the characteristics of the workload.

The figure Figure 13 shows that PowerTrip consistently outperforms all baseline policies across all tested update sizes. For the small (1GB) update, local computation speed is more important than the communication delay, thus ($w_p > w_c$) and PowerTrip achieves a time-to-accuracy of 0.94 hours. As the update size increases to 2GB, the communication delay is the primary bottleneck, the ($w_c > w_p$) and PowerTrip's time-to-accuracy only modestly increases to 1.02 hours, still giving 23.3% better time-to-accuracy than the static and centralized baselines (both 1.33 hours). This

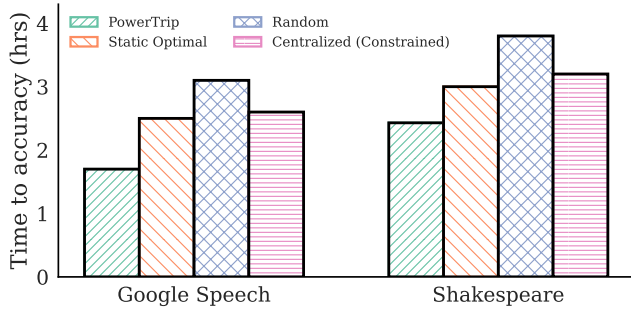


Figure 14: Time-to-accuracy comparison across different datasets, demonstrating that PowerTrip consistently outperforms all baseline policies and highlighting the robustness.

confirms that PowerTrip’s adaptive selection strategy is effective for a wide range of training workloads.

Performance Across Diverse Datasets. We repeat our end-to-end training experiments on two additional, widely-used datasets: the Shakespeare dataset for next-character prediction (using an LSTM) and the Google speech commands dataset for keyword spotting (using a KWT-1 model). The results confirm that PowerTrip consistently performs better across different data modalities and model architectures. As shown in Figure 14, on the Google speech commands task, PowerTrip achieves the target accuracy 32% faster than the static optimal policy for that workload. Similarly, on the Shakespeare dataset, our dynamic policy still outperforms the static baseline by 19%. This demonstrates that our dynamic, greedy heuristic is a robust approach that effectively maximizes the performance across a variety of common ML tasks.

Key point. PowerTrip’s adaptive selection strategy proves effective across a wide range of training workloads, demonstrating its robustness on different datasets and models.

6 Related Work

Our work is positioned at the intersection of distributed machine learning, resource management, and performance optimization. Below, we review prior work to show how PowerTrip differs.

Communication-Efficient Distributed and Federated Learning. The challenge of communication overhead in distributed training is well-established [13, 25, 40]. A significant body of research has focused on reducing the volume of data that must be transmitted during synchronization [4, 25, 29]. Techniques such as gradient quantization [4], where gradients are represented with fewer bits, and sparsification [3], where only a subset of significant updates are sent, have proven effective. More recently, parameter-efficient fine-tuning (PEFT) methods like Low-Rank Adaptation (LoRA) [21] and QLoRA [15] have become standard for distributed LLM training. Systems like DiLoCo [17] and Photon [38] leverage these PEFT techniques to enable training across many distributed clients with substantially lower bandwidth requirements. Similarly, FusionLLM [44] introduces adaptive compression methods to further optimize communication in decentralized settings. While these approaches are critical, they primarily optimize *what* is sent, not *who* sends it. They reduce data volume but do not dynamically select participants based on their power, compute, and network resources. In

contrast, PowerTrip introduces a higher-level optimization layer that decides which client sites should participate in the distributed ML training to maximize overall system performance.

Resource-Constrained AI Training. Prior work addresses the growing resource constraints in AI [33, 41, 42]. Research on “Green AI” has focused on developing models and training methods that are more computationally and energy-efficient [39]. This includes creating more efficient model architectures and algorithms that require fewer floating-point operations to converge. Prior work schedules training jobs to align with periods of high renewable energy availability or low electricity prices [47].

SkyPilot [48], an inter-cloud broker provisions limited GPU resources across different cloud providers, optimizing for cost and availability. Prior work [43] also analyzes cross-region training as a solution to single-site GPU shortages. The work confirms that while aggregating resources across regions is beneficial, the performance is often bottlenecked by high inter-region network latency, highlighting the same core trade-off we address. These works assume a constant homogeneous power supplies and ignore the challenge of heterogeneous power availability between sites.

Systems above address resource scarcity but focus on different aspects. Green AI focuses on algorithmic efficiency, while SkyPilot focuses on resource provisioning and cost before a job starts. Prior work hasn’t addressed optimizing distribution to maximize performance gains for large-scale AI training workloads.

Dynamic Scaling and Participant Selection Policies [12, 18]. The concept of dynamic participant selection is well-explored in distributed machine learning. Prior systems like EcoLearn [30], FedCS [31], and Oort [26] select clients to optimize for specific goals such as carbon footprint, training deadlines, or statistical utility. While effective, these methods typically focus on a single dimension of the problem, such as data quality or energy use [10, 28]. None of this prior work proposes a selection strategy that holistically optimizes for end-to-end training time by dynamically balancing the aggregate computational power of a set of sites against their combined communication cost. Furthermore, many existing scaling policies can be reactive [2, 30]. In contrast, PowerTrip uses a proactive, bottom-up greedy policy guided by a single objective: maximizing training efficiency in power-constrained environments.

7 Conclusion

This paper addresses the problem of optimizing the computational throughput and communication latency in geo-distributed, large-scale AI training to overcome the power and resource limitations of a single site. We introduce PowerTrip, a system that dynamically optimizes the number of training sites at runtime by implementing a greedy heuristic to identify the point where adding more sites becomes detrimental to performance (time-to-accuracy). Our evaluation demonstrates that by exploiting this trade-off, PowerTrip can reduce the time-to-accuracy by up to 50% compared to existing baselines. Future work will involve deploying PowerTrip on a public cloud to validate its performance in a real-world setting and extending its optimization heuristic to consider the monetary costs of compute instances and data egress.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren-
cia Leoni Aleman, Diogo Almeida, Janko Altmerschmidt, Sam Altman, Shyamal
Anadkat, et al. Gpt-4 Technical Report. *arXiv*, 2023.
- [2] Saurabh Agarwal, Hongyi Wang, Kangwook Lee, Shivaram Venkataraman, and
Dimitris Papailiopoulos. Accordion: Adaptive Gradient Communication via
Critical Learning Regime Identification. In *Proceedings of Machine Learning and
Systems*, 2021.
- [3] Alham Fikri Aji and Kenneth Heafield. Sparse Communication for Distributed
Gradient Descent. *arXiv*, 2017.
- [4] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic.
QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding.
NEURIPS, 2017.
- [5] Victor Avelar, Patrick Donovan, Paul Lin, Wendy Torell, and Maria A Torres
Arango. The AI Disruption: Challenges and Guidance for Data Center Design.
Artificial Intelligence in Medicine, 2023.
- [6] Axel Berg, Mark O'Connor, and Miguel Tairum Cruz. Keyword Transformer: A
Self-Attention Model for Keyword Spotting. *arXiv*, 2021.
- [7] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-
Marques, Yan Gao, Lorenzo Sani, Hei Li Kwing, Titouan Parcollet, Pedro PB de
Gusmão, and Nicholas D Lane. Flower: A Friendly Federated Learning Research
Framework. *arXiv*, 2020.
- [8] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex
Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi,
Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and
Jason Roselander. Towards Federated Learning at Scale: System Design. In
MLSYS, 2019.
- [9] Sebastian Caldas, Sai Meher Karthik Duddu, Peter Wu, Tian Li, Jakub Konečný,
H Brendan McMahan, Virginia Smith, and Ameet Talwalkar. Leaf: A Benchmark
for Federated Settings. *arXiv*, 2018.
- [10] Hyunsung Cho, Akhil Mathur, and Fahim Kawsar. FLAME: Federated Learning
across Multi-device Environments. *Proceedings of the ACM on Interactive, Mobile,
Wearable and Ubiquitous Technologies*, 2022.
- [11] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. EMNIST:
Extending MNIST to Handwritten Letters. In *Neural Networks*, 2017.
- [12] Gergely Dániel Németh, Miguel Ángel Lozano, Novi Quadrianto, and Nuria
Oliver. A Snapshot of the Frontiers of Client Selection in Federated Learning.
arXiv, 2022.
- [13] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao,
Marc'aurilio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. Large Scale
Distributed Deep Networks. *NEURIPS*, 2012.
- [14] Peter J Denning and Ted G Lewis. Exponential Laws of Computing Growth.
Commun. ACM, 2016.
- [15] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA:
Efficient Finetuning of Quantized LLMs. *NEURIPS*, 2023.
- [16] Haotian Dong, Jingyan Jiang, Rongwei Lu, Jiajun Luo, Jiajun Song, Bowen Li,
Ying Shen, and Zhi Wang. Beyond A Single AI Cluster: A Survey of Decentralized
LLM Training. *arXiv*, 2025.
- [17] Arthur Douillard, Qixuan Feng, Andrei A Rusu, Rachita Chhaparia, Yani Donchev,
Adhiguna Kuncoro, Marc'aurilio Ranzato, Arthur Szlam, and Jiajun Shen.
DiLoCo: Distributed Low-Communication Training of Language Models. *arXiv*,
2023.
- [18] Lei Fu, Huanle Zhang, Ge Gao, Mi Zhang, and Xin Liu. Client Selection in
Federated Learning: Principles, Challenges, and Opportunities. *Internet of Things
Journal*, 2023.
- [19] Google. TensorFlow Federated: Machine Learning on Decentralized Data. <https://www.tensorflow.org/federated>, 2019.
- [20] Google. Efficiency: How We Do It. <https://www.google.com/about/datacenters/efficiency/>, 2024.
- [21] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean
Wang, Lu Wang, Weizhu Chen, et al. LoRA: Low-Rank Adaptation of Large
Language Models. *ICLR*, 2022.
- [22] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Taiyi Wang, Bin Cui, Ana Klimovic,
and Eiko Yoneki. ThunderServe: High-Performance and Cost-Efficient LLM
Serving in Cloud Environments. *arXiv*, 2025.
- [23] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang,
Yanhua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. MegaScale: Scaling Large
Language Model Training to More than 10,000 GPUs. *NSDI*, 2024.
- [24] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess,
Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling
Laws for Neural Language Models. *arXiv*, 2020.
- [25] Jakub Konečný, H Brendan McMahan, Felix X Yu, Peter Richtárik,
Ananda Theertha Suresh, and Dave Bacon. Federated Learning: Strategies for
Improving Communication Efficiency. *arXiv*, 2016.
- [26] Fan Lai, Xiangfeng Zhu, Harsha V Madhyastha, and Mosharaf Chowdhury. Oort:
Efficient Federated Learning via Guided Participant Selection. In *USENIX*, 2021.
- [27] Vivian Lee, Pattabi Seshadri, Clark O'Neill, Archit Choudhary, Braden Holstege,
and Stefan A Deutscher. Breaking barriers to data center growth. *Boston Con-
sulting Group*, 2025.
- [28] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and
Virginia Smith. Federated Optimization in Heterogeneous Networks. *Proceedings
of Machine Learning and Systems*, 2020.
- [29] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J Dally. Deep Gradient
Compression: Reducing the Communication Bandwidth for Distributed Training.
arXiv, 2017.
- [30] Talha Mehboob, Noman Bashir, Jesus Omana Iglesias, Michael Zink, and David
Irwin. Ecolearn: Optimizing the Carbon Footprint of Federated Learning. *arXiv*,
2023.
- [31] Takayuki Nishio and Ryo Yonetani. Client Selection for Federated Learning with
Heterogeneous Resources in Mobile Edge. In *ICC*, 2019.
- [32] Tyler Norris, Timothy Profeta, Dalia Patino-Echeverri, and Adam Cowie-Haskell.
Rethinking Load Growth: Assessing the Potential for Integration of Large Flexible
Loads in US Power Systems. 2025.
- [33] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia,
Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon Emissions and
Large Neural Network Training. *arXiv*, 2021.
- [34] AI Power. Expanding Data Center Capacity to Meet Growing Demand. *McKinsey
and Company*, 2024.
- [35] Theo Ryffel, Andrew Trask, Morten Dahl, Bobby Wagner, Jason Mancuso, Daniel
Rueckert, and Jonathan Passerat-Palmbach. A Generic Framework for Privacy
Preserving Deep Learning. *arXiv*, 2018.
- [36] Goldman Sachs. AI, Data Centers and the Coming US Power De-
mand Surge. [https://www.goldmansachs.com/pdfs/insights/pages/generational-
growth-ai-data-centers-and-the-coming-us-power-surge/report.pdf](https://www.goldmansachs.com/pdfs/insights/pages/generational-growth-ai-data-centers-and-the-coming-us-power-surge/report.pdf), 2024.
- [37] Varun Sakalkar, Vasileios Kontorinis, David Landhuis, Shaohong Li, Darren
De Ronde, Thomas Blooming, Anand Ramesh, James Kennedy, Christopher
Malone, Jimmy Clidaras, et al. Data Center Power Oversubscription with a
Medium Voltage Power Plane and Priority-Aware Capping. In *ASPLOS*, 2020.
- [38] Lorenzo Sani, Alex Jacob, Zeyu Cao, Royson Lee, Bill Marino, Yan Gao, Dongqi
Cai, Zexi Li, Wanru Zhao, Xinchu Qiu, et al. Photon: Federated LLM Pre-Training.
arXiv, 2024.
- [39] Roy Schwartz, Jesse Dodge, Noah A Smith, and Oren Etzioni. Green AI. *Com-
munications*, 2020.
- [40] Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Chengjian Liu, Wei Wang, and
Bo Li. A Quantitative Survey of Communication Optimizations in Distributed
Deep Learning. *IEEE Network*, 2020.
- [41] Tianyao Shi, Yanran Wu, Sihang Liu, and Yi Ding. GreenLLM: Disaggregating
Large Language Model Serving on Heterogeneous GPUs for Lower Carbon
Emissions. *arXiv*, 2024.
- [42] Siddharth Singh, Prajwal Singhania, Aditya Ranjan, John Kirchenbauer, Jonas
Geiping, Yuxin Wen, Neel Jain, Abhimanyu Hans, Manli Shu, Aditya Tomar,
et al. Democratizing AI: Open-Source Scalable LLM Training on GPU-Based
Supercomputers. In *SC24*, 2024.
- [43] Foteini Strati, Paul Elvinger, Tolga Kerimoglu, and Ana Klimovic. ML Training
with Cloud GPU Shortages: Is Cross-Region the Answer? In *MLSys*, 2024.
- [44] Zhenheng Tang, Xueze Kang, Yiming Yin, Xinglin Pan, Yuxin Wang, Xin He,
Qiang Wang, Rongfei Zeng, Kaiyong Zhao, Shaohuai Shi, et al. Fusionllm: A
Decentralized LLM Training System on Geo-Distributed GPUs with Adaptive
Compression. *arXiv*, 2024.
- [45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yas-
mine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrubti Bhos-
ale, et al. LLaMa 2: Open Foundation and Fine-Tuned Chat Models. *arXiv*,
2023.
- [46] Pete Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech
Recognition. *arXiv*, 2018.
- [47] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani,
Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, et al. Sus-
tainable AI: Environmental Implications, Challenges and Opportunities. *MLSys*,
2022.
- [48] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj,
Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker,
et al. SkyPilot: An Intercloud Broker for Sky Computing. In *NSDI*, 2023.
- [49] Rui Ye, Wenhao Wang, Jingyi Chai, Dihan Li, Zexi Li, Yinda Xu, Yaxin Du,
Yanfeng Wang, and Siheng Chen. Openfedllm: Training Large Language Models
on Decentralized Private Data via Federated Learning. In *SIGKDD*, 2024.
- [50] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui
Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. OPT: Open
Pre-Trained Transformer Language Models. *arXiv*, 2022.